

1. Objectives:

1. To understand the fundamental concept of sorting algorithms in data structures.
2. To understand Bubble Sort, Selection Sort, and Insertion Sort algorithms.
3. To know the working procedure, efficiency, and limitations of sorting techniques.

2. Introduction:

- **Sorting:** Sorting is one of the most fundamental operations in computer science. It refers to the process of arranging data elements in a specific order, typically ascending or descending. Sorting improves data organization and makes other operations such as searching, merging, and analysis more efficient. In real-world applications like databases, search engines, and data processing systems, sorting plays a crucial role in performance optimization. Some sorting techniques are Bubble Sort, Selection Sort, Insertion Sort.
- **Bubble Sort:** Bubble Sort is a simple comparison-based sorting algorithm that repeatedly traverses the array and compares adjacent elements, swapping them if they are in the wrong order. The algorithm works through multiple passes over the array. In the first pass, the largest element moves to the last position. In the second pass, the second largest element moves to the second last position, and this process continues. After k passes, the largest k elements are placed correctly at the end of the array. Because larger elements gradually “bubble up” to their correct positions through repeated swaps, the algorithm is named Bubble Sort.

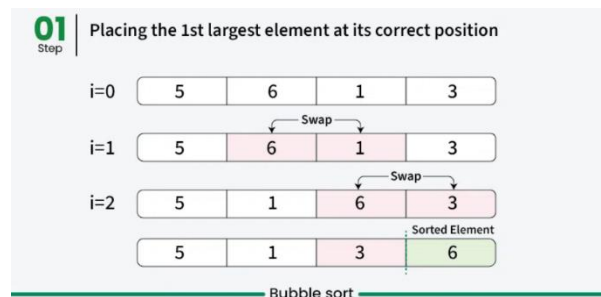


Fig:4.1: Bubble Sort

- **Selection Sort:** Selection Sort is another comparison-based sorting algorithm that divides the array into two parts: a sorted portion and an unsorted portion. Initially, the sorted portion is empty, and the entire array is considered unsorted. In each iteration, the algorithm selects the smallest element from the unsorted portion and swaps it with the first element of that portion. This process continues until all elements are placed in their correct positions. Unlike Bubble Sort, Selection Sort performs fewer swaps because only one swap is done per pass. Selection Sort is not efficient for large datasets.

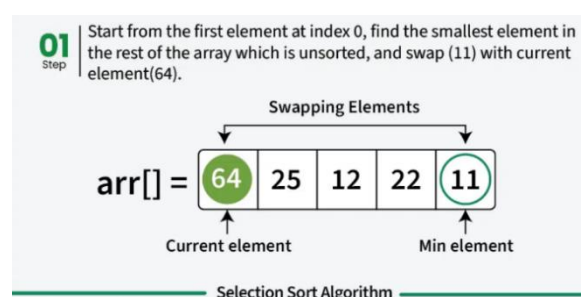


Fig:3.2: Binary Search

- **Insertion Sort:** Insertion Sort is a comparison-based sorting algorithm that builds the sorted array gradually by inserting each element into its proper position. It works similarly to arranging playing cards in hand. The array is divided into a sorted portion and an unsorted portion. In each step, one element from the unsorted portion is picked and inserted into the correct position within the sorted portion by shifting larger elements to the right. This shifting continues until the correct position is found. Insertion Sort is efficient for small datasets and performs particularly well when the array is already nearly sorted. In such cases, its best-case time complexity is $O(n)$. It is simple, stable, and practical for small-scale applications.

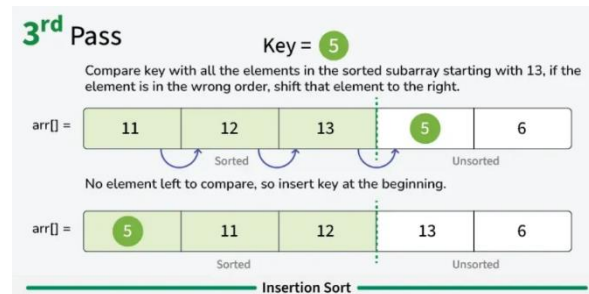


Fig:4.3: Insertion Sort

3. Procedure

Experiment 1: Bubble Sort

Input:

- $v[]$ – A vector containing integer elements
- n – Size of the vector

Output:

- Sorted elements of the vector in ascending order

Algorithm:

1. **for** $i \leftarrow 0$ to $n - 1$ **do**
2. **for** $j \leftarrow 0$ to $n - i - 2$ **do**
3. **if** $v[j] > v[j + 1]$ **then**
4. $t \leftarrow v[j]$
5. $v[j] \leftarrow v[j + 1]$
6. $v[j + 1] \leftarrow t$
7. **for** $i \leftarrow 0$ to $n - 1$ **do**
8. **print** $v[i]$

Experiment 2: Selection Sort

Input:

- $v[]$ – A vector containing integer elements
- n – Size of the vector

Output:

- Sorted elements of the vector in ascending order

Algorithm:

1. **for** $i \leftarrow 0$ to $n - 1$ **do**

```

2.    m ← v[i]
3.    k ← i
4.    for j ← i to n - 1 do
5.        if v[j] < m then
6.            m ← v[j]
7.            k ← j
8.    t ← v[i]
9.    v[i] ← v[k]
10.   v[k] ← t
11. for i ← 0 to n - 1 do
12.   print v[i]

```

Experiment 3: Insertion Sort

Input:

- v[] – A vector containing integer elements
- n – Size of the vector

Output:

- Sorted elements of the vector in ascending order

Algorithm:

```

1. for i ← 1 to n - 1 do
2.   j ← i - 1
3.   t ← v[i]
4.   while j ≥ 0 and t < v[j] do
5.       v[j + 1] ← v[j]
6.       j ← j - 1
7.   v[j + 1] ← t
8. for i ← 0 to n - 1 do
9.   print v[i]

```

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Bubble Sort

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cout << "Input Size: ";
    cin >> n;
    cout << "Input Elements: ";

    vector<int> v;
    for (int i = 0; i < n; i++){
        int k;
        cin >> k;
        v.push_back(k);
    }
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n - i - 1; j++){
            if (v[j] > v[j + 1]){
                int t = v[j];
                v[j] = v[j + 1];
                v[j + 1] = t;
            }
        }
    }
}

```

```

        v[j] = v[j + 1];
        v[j + 1] = t;
    }
}
for (int i = 0; i < n; i++){
    cout << v[i] << " ";
}
return 0;
}

```

Table 4.1: Experiment 1 output

Input	Expected Output	Actual Output
Input Size: 5 Input Elements: 3 7 1 9 4	Input Size: 5 Input Elements: 3 7 1 9 4 1 3 4 7 9	Input Size: 5 Input Elements: 3 7 1 9 4 1 3 4 7 9

Experiment 2: Selection Sort

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cout << "Input Size: ";
    cin >> n;
    cout << "Input Elements: ";

    vector<int> v;
    for (int i = 0; i < n; i++){
        int k;
        cin >> k;
        v.push_back(k);
    }

    for (int i = 0; i < n; i++){
        int m = v[i];
        int k = i;
        for (int j = i; j < n; j++){
            if (v[j] < m){
                m = v[j];
                k = j;
            }
        }
        int t = v[i];
        v[i] = v[k];
        v[k] = t;
    }

    for (int i = 0; i < n; i++){
        cout << v[i] << " ";
    }
    return 0;
}

```

Table 4.2: Experiment 2 output

Input	Expected Output	Actual Output
Input Size: 7 Input Elements: 2 5 3 1 9 4 6	Input Size: 7 Input Elements: 2 5 3 1 9 4 6 1 2 3 4 5 6 9	Input Size: 7 Input Elements: 2 5 3 1 9 4 6 1 2 3 4 5 6 9

Experiment 3: Insertion Sort

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cout << "Input Size: ";
    cin >> n;
    cout << "Input Elements:";

    vector<int> v;
    for (int i = 0; i < n; i++){
        int k;
        cin >> k;
        v.push_back(k);
    }

    for (int i = 1; i < n; i++){
        int j = i - 1;
        int t = v[i];
        while (j >= 0 && t < v[j]){
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = t;
    }

    for (int i = 0; i < n; i++){
        cout << v[i] << " ";
    }
    return 0;
}

```

Table 4.3: Experiment 3 output

Input	Expected Output	Actual Output
Input Size: 10 Input Elements: 8 3 5 2 9 1 4 7 6 0	Input Size: 10 Input Elements: 8 3 5 2 9 1 4 7 6 0 0 1 2 3 4 5 6 7 8 9	Input Size: 10 Input Elements: 8 3 5 2 9 1 4 7 6 0 0 1 2 3 4 5 6 7 8 9

6. Discussion & Conclusion:

In this lab, we successfully learned three fundamental sorting algorithms: Bubble Sort, Selection Sort, and Insertion Sort. We observed that all three algorithms are suitable for small datasets, but their efficiency decreases significantly as the size of the data increases due to their $O(n^2)$ time complexity. Bubble Sort repeatedly swaps adjacent elements, Selection Sort minimizes swaps by selecting the minimum element in each pass, and Insertion Sort efficiently shifts elements to insert them into the correct position. While implementing these algorithms, we faced some errors maintaining loop boundaries, indexing, and swapping logic. Through practical implementation, we learned how different sorting techniques operate, their advantages and limitations, and when each method is appropriate. Overall, this experiment strengthened our understanding of sorting mechanisms and their importance in efficient data management and algorithm design.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/> accessed at 6.25 PM on 26th February 2026.
2. <https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/> accessed at 6.40 PM on 26th February 2026.
3. <https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/> accessed at 9.35 AM on 27th February 2026.